

PROJECT REPORT

Digital Timer with Event Scheduler

using PIC16F877A, DS1307 RTC and 16x2 LCD

Embedded C | I2C | EEPROM | Timer Interrupt

Submitted by	
Register / ID number	
Department / Organization	
Institution	
Guide / Mentor	
Date	September 2026

Contents

1. Abstract	3
2. Introduction	3
3. System Overview	3
4. Hardware Description	4
5. Software Design	5
6. Working Principle	7
7. Implementation Details	8
8. Testing and Debugging	8
9. Applications	9
10. Advantages and Limitations	9
11. Future Scope	9
12. Conclusion	10
13. References	10

1. Abstract

This report describes the design and implementation of a digital timer with an event scheduler using the PIC16F877A 8-bit microcontroller. The system reads the real time and date from a DS1307 real-time clock (RTC) chip through the I2C protocol and shows them on a 16x2 character LCD. With six push buttons, the user can move through a menu to set the time, set the date and store up to three events. Each event has a start time, a duration and a frequency (once, daily or weekly). The events are saved in the internal EEPROM, so they are kept when the power is removed. A Timer1 interrupt is used to switch the display between the date/time screen and the event screen.

The firmware is written in embedded C with MPLAB X IDE and the XC8 compiler. Drivers for the LCD, the I2C bus, the DS1307 and the keypad were written from scratch. The report explains the hardware, the software design, the main algorithms, the memory usage, and the problems that were solved during development (a RAM overflow error and a blank display caused by unhandled interrupts). It also lists applications and future improvements.

2. Introduction

Many electronic products need to know the time and to act on a schedule: medicine reminders, irrigation controllers, street light timers, industrial machine schedulers and school bell systems. A microcontroller alone cannot keep time when it is powered off or reset. An RTC chip with its own crystal and battery solves this problem, and a non-volatile memory keeps the user settings.

2.1 Problem statement

To build a low-cost, stand-alone device that keeps accurate date and time, lets the user configure time, date and timed events using a small display and a few buttons, and keeps the saved events after a power failure.

2.2 Objectives

1. Interface the DS1307 RTC with the PIC16F877A using the I2C protocol and display date and time.
2. Develop a 16x2 LCD driver in 8-bit mode.
3. Design a menu-driven user interface with six push buttons.
4. Store and read events (start time, duration, frequency) using the internal EEPROM.
5. Use a Timer1 interrupt for periodic timing without blocking the main loop.
6. Fit the complete firmware inside the small memory of the PIC16F877A (368 bytes of RAM).

3. System Overview

The PIC16F877A is the central controller. It reads the time and date from the DS1307 over I2C, reads the six push buttons on PORTB, drives the LCD on PORTD, RE1 and RE2, and saves events in its internal EEPROM. Timer1 gives a time base of about one second.

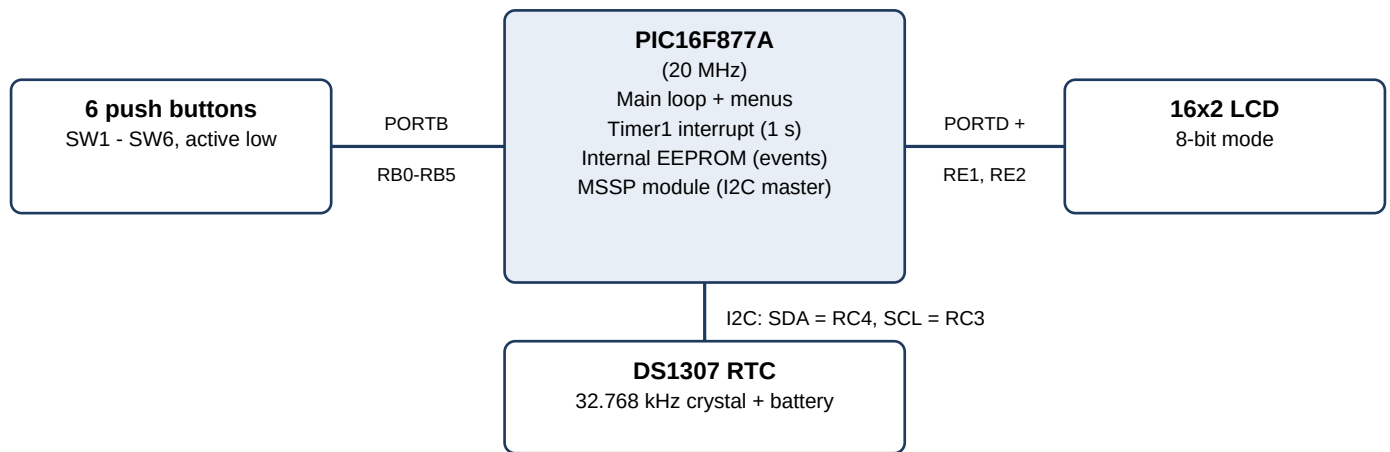


Figure 1: Block diagram of the system

3.1 Main features

- Live date and time display from the DS1307 RTC.
- Menu-driven interface with six push buttons.
- Set time and set date, with a blinking field cursor.
- Set up to three events: start time, duration and frequency (Once, Daily, Weekly).
- View saved events one by one.
- Events stored in EEPROM, so they stay after power off.
- Dashboard switches between date/time (5 s) and the saved event (2 s).

4. Hardware Description

4.1 Components

Component	Description	Role in the project
PIC16F877A	8-bit RISC microcontroller, 8K words program memory, 368 bytes RAM, 256 bytes EEPROM, MSSP module (I2C), three timers	Main controller
DS1307 RTC	Serial real-time clock, I2C interface, BCD time and date, 32.768 kHz crystal, battery backup	Keeps time and date
16x2 LCD	HD44780-compatible character display, 8-bit data mode	User display
Push buttons (6)	Active-low switches on PORTB	Menu control
Crystal (20 MHz)	Main oscillator	Instruction clock 5 MHz ($F_{osc}/4$)
Pull-up resistors	4.7 kohm on SDA and SCL	I2C bus idle level

4.2 Pin connections

Signal	PIC pin	Connected to
LCD data D0 - D7	RD0 - RD7 (PORTD)	LCD data lines
LCD RS	RE2	Register select
LCD EN	RE1	Enable
LCD RW	-	Tied to GND (driver only writes)
I2C SDA	RC4	DS1307 SDA (with pull-up)
I2C SCL	RC3	DS1307 SCL (with pull-up)
SW1 - SW6	RB0 - RB5	Push buttons, active low (pressed = 0)

4.3 DS1307 registers used

The DS1307 has the I2C address 0x68. The firmware uses 0xD0 for write and 0xD1 for read. All values are stored in BCD.

Address	Register	Notes
0x00	Seconds	Bit 7 is the Clock Halt (CH) bit. It must be 0 for the clock to run.
0x01	Minutes	Tens digit in bits 6-4
0x02	Hours	24-hour format is used (tens digit in bits 5-4)
0x04	Date	Day of the month
0x05	Month	1 to 12
0x06	Year	Two digits (00 - 99)

4.4 LCD interface

The LCD works in 8-bit mode. The start-up sequence sends 0x33 (three times), then 0x38 (2 lines, 5x8 font), 0x01 (clear) and 0x0C (display on, cursor off). Line 1 starts at address 0x80 and line 2 at 0xC0, so the cursor position is $0x80 + x$ or $0xC0 + x$. The EN pulse is 100 us and each write waits 4.1 ms.

5. Software Design

5.1 Development tools

MPLAB X IDE, XC8 compiler (v4.00) and the PIC16Fxxx device family pack (1.2.33). The code is written in C99.

5.2 Modules

File	Description
main.c	Initialization and main loop
digital_timer.c / .h	Dashboard, menus, set time, set date, set event, view event
clcd.c / .h	LCD driver: init_clcd(), clcd_print()
i2c.c / .h	I2C master driver: start, repeated start, stop, read, write
ds1307.c / .h	RTC driver: init_ds1307(), read_ds1307(), write_ds1307()
digital_keypad.c, newfile.h	Keypad reading (LEVEL and STATE mode), SW1 - SW6 macros
TIMER.c / Timer.h	Timer setup
isr.c	Timer1 interrupt service routine

5.3 Program flow

The main program initializes all peripherals and then runs an endless loop. In every loop cycle it reads a key. If the menu is open, or SW1 or SW2 is pressed on the dashboard, the menu handler runs. Otherwise the dashboard is drawn. The Timer1 interrupt runs in the background and only updates counters and one display flag.

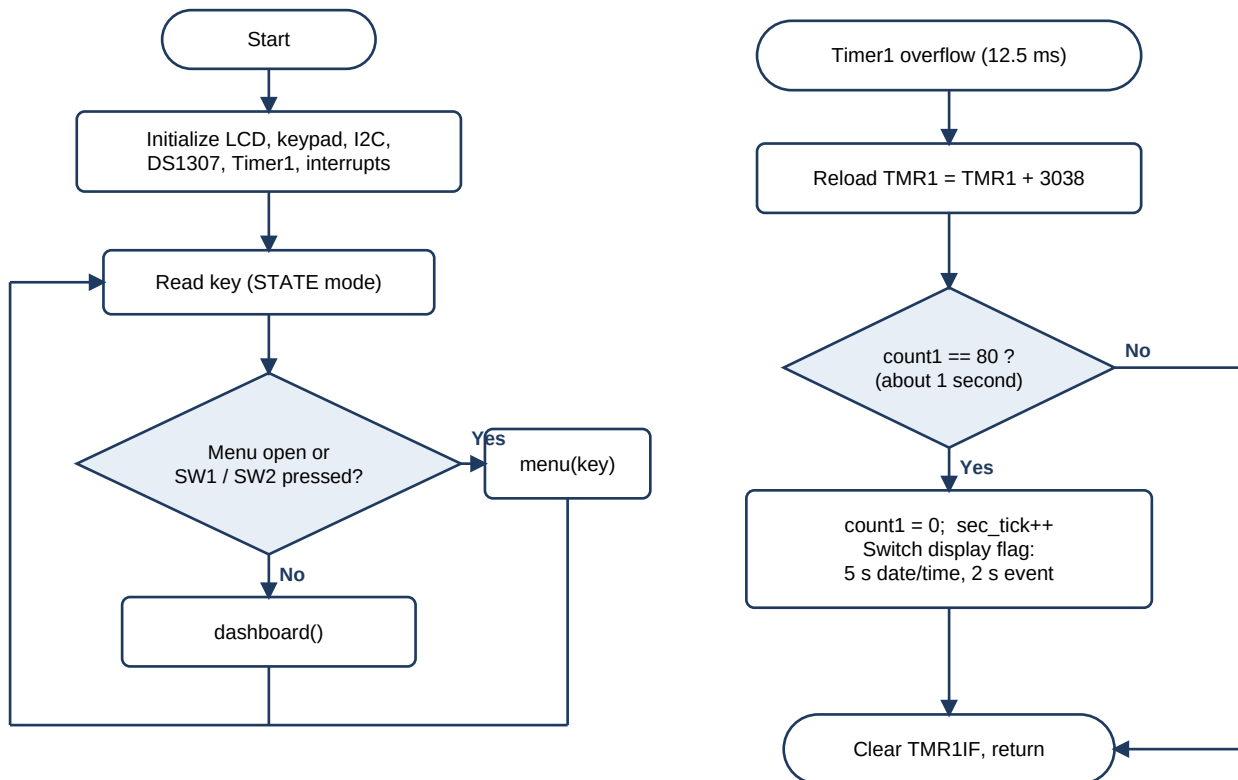


Figure 2: Main program flowchart (left) and Timer1 interrupt flowchart (right)

5.4 Menu structure and button functions

```
Dashboard --(SW1 or SW2)--> Main menu
SET/VIEW EVNT --> SET EVENT / VIEW EVENT
SET TIME/DATE --> SET TIME / SET DATE
SW6 = go back one level
```

Button	Value read from PORTB	Function
SW1	0x3E	Move up / decrease value
SW2	0x3D	Move down / increase value
SW3	0x3B	Select / next field
SW4	0x37	Save time, date or event
SW6	0x1F	Back / cancel

The menu is a state machine. Flag variables such as menu_inside, event_menu_inside and date_time_inside show which screen is active. Each screen function handles its own keys and returns to the parent screen on SW6.

5.5 Event storage in EEPROM

Each event uses 5 bytes. The address of an event is the event number x 5. The maximum is three events (15 bytes).

Byte	0	1	2	3	4
Data	Start hour	Start minute	Duration hour	Duration minute	Frequency (0 = once, 1 = daily, 2 = weekly)

5.6 Key algorithms

BCD conversion. The DS1307 returns BCD values. Each digit is extracted with a shift and a mask. Before writing, a decimal value is converted back to BCD.

```
/* BCD to ASCII digits (seconds) */
time_str[11] = ((clock_reg[2] >> 4) & 0x07) + '0';
time_str[12] = (clock_reg[2] & 0x0F) + '0';

/* Decimal to BCD */
bcd_min = ((dt_min / 10) << 4) | (dt_min % 10);
```

Key press detection (STATE mode). The function returns a key only once per press. The flag once blocks repeated reads until all keys are released. A short delay gives debounce.

```
if ((PORTB & 0x3F) != ALL_RELEASED && once)
{
    once = 0;
    for (int i = 500; i--; );
    return (PORTB & 0x3F);
}
else if ((PORTB & 0x3F) == ALL_RELEASED)
    once = 1;
```

Non-blocking blink. While setting a value, a counter toggles blink_state every few loop cycles and the selected field is printed as spaces when the state is 1. No long delay is used, so the buttons are still read.

6. Working Principle

1. At power-up, `init_config()` initializes the LCD, keypad, I2C, DS1307 and Timer1, and enables the interrupts.
2. **init_ds1307()** reads the seconds register and clears the CH bit, so the RTC oscillator runs.
3. The main loop reads a key in STATE mode.
4. On the dashboard, the time and date are read from the DS1307, converted from BCD to characters and printed on the LCD.
5. Every 12.5 ms the Timer1 ISR runs. After 81 overflows (about 1 second) it increases `sec_tick`. After 5 s the display flag is set and the dashboard shows the time and the first saved event from EEPROM. After 2 s the flag is cleared and the date and time are shown again.
6. SW1 or SW2 opens the main menu. The cursor (->) moves with SW1 and SW2 and SW3 selects.
7. **Set time / set date:** the current value is read from the DS1307. The user edits each field and SW4 converts the values to BCD and writes them to the DS1307.
8. **Set event:** the user edits start time, duration and frequency. SW4 writes the 5 bytes to EEPROM and increases the event count. A fourth event is refused with the message MAX EVENTS SET.
9. **View event:** the 5 bytes of the selected event are read from EEPROM and shown on the LCD. SW1 and SW2 move between events.

7. Implementation Details

7.1 Timer1 and I2C calculations

Item	Calculation	Result
Instruction clock	$F_{osc} / 4 = 20 \text{ MHz} / 4$	5 MHz (0.2 us per count)
Timer1 counts per overflow	65536 - 3036 (preload)	62500 counts
Timer1 overflow period	$62500 \times 0.2 \text{ us}$	12.5 ms
One-second tick	81 overflows $\times$ 12.5 ms	about 1.01 s
I2C clock (100 kHz)	$SSPADD = F_{osc} / (4 \times \text{baud}) - 1 = 20 \text{ MHz} / (4 \times 100 \text{ kHz}) - 1$	49

7.2 Memory usage

The values below come from the MPLAB X memory summary of a successful build. They may change slightly when the code is edited.

Memory	Used	Available	Usage
Program memory	6302 words (0x189E)	8192 words (0x2000)	76.9 %
Data memory (RAM)	349 bytes (0x15D)	368 bytes (0x170)	94.8 %
EEPROM	0 bytes at build time	256 bytes	Events use up to 15 bytes at run time

8. Testing and Debugging

8.1 Functional test cases

No.	Test	Procedure	Expected result
1	LCD initialization	Print HELLO after init_clcd()	Text appears on line 1
2	RTC read	Power on with the DS1307 running	Date and time shown; seconds increase
3	Menu navigation	Press SW1 or SW2, then SW3, then SW6	Correct screens; SW6 goes back
4	Set time	Edit HH:MM:SS, press SW4	TIME SAVED! shown; new time on dashboard
5	Set date	Edit DD-MM-YY, press SW4	DATE SAVED! shown; new date on dashboard
6	Set event	Edit start, duration, frequency, press SW4	EVENT SAVED!; the fourth event shows MAX EVENTS SET
7	View event	Open VIEW EVENT, use SW1 and SW2	Events shown one by one; NO EVENTS SAVED when empty
8	Display switching	Stay on the dashboard with one event saved	Date/time for 5 s, then time and event for 2 s
9	Key handling	Hold a button down	Only one action per press

After the fixes described below, the complete program was loaded on the hardware and worked as intended.

8.2 Problems and solutions

Problem	Cause	Solution
Build error: could not find space (1 byte) for variable set_time@dt_min	The 368 bytes of data RAM were full because of unused and repeated arrays	Removed unused global buffers, declared repeated arrays once and used one shared delay function. Final RAM use is 349 of 368 bytes.
LCD blank after loading the hex file (a HELLO test worked)	Timer0 and Timer2 interrupts were enabled, but the ISR handled only Timer1. Their flags were never cleared, so the ISR ran again and again and main() got almost no time	Removed the init_timer0() and init_timer2() calls. Every enabled interrupt needs a handler that clears its flag.

9. Applications

- Medicine reminder box (pill time, daily repeat).
- Industrial pump or motor scheduler (start at a set time for a set duration).
- Street light and sign board timer.
- Irrigation controller (daily or weekly watering).
- School or college bell system.
- Home appliance timers (washing machine, geyser, air conditioner).
- Data logger or smart meter with time stamps.

10. Advantages and Limitations

10.1 Advantages

- Low cost and simple hardware.
- Accurate timekeeping, and the time continues on the DS1307 battery when the board is off.
- Events are kept in non-volatile memory.
- Modular code: each driver is in its own file and can be reused.
- Non-blocking design: timing uses an interrupt and blinking uses a counter.

10.2 Limitations

- The events are stored and displayed only. No output (relay or LED) is switched yet.
- The event count and write address are kept in RAM, so they start from zero after a reset.
- The dashboard shows only the first saved event.
- The date is not checked for impossible values (for example 31 February).
- The I2C wait loops have no time-out, so the code can hang if the DS1307 is missing.
- The LCD driver waits 4.1 ms after every character, so the screen update is slow.

11. Future Scope

- Compare the RTC time with the event start time and switch a relay for the event duration.
- Save the event count in EEPROM and restore it at start-up.
- Show the next upcoming event on the dashboard and add a delete-event option.
- Add a buzzer alarm and a weekday selection for weekly events.
- Declare the variables shared with the ISR as volatile and add I2C time-outs.
- Add UART or Bluetooth for setting time and events from a phone or PC.

- Use interrupt-on-change on PORTB for the keypad to save CPU time.

12. Conclusion

A digital timer with an event scheduler was designed and built with the PIC16F877A. The project combines several important embedded topics: I2C communication with an RTC, BCD handling, an 8-bit LCD driver, EEPROM storage, Timer1 interrupts, keypad edge detection and a menu state machine. Development also showed the value of careful RAM use on a small device and of writing a handler for every enabled interrupt. The design can be extended to a complete scheduler by adding an output stage that acts on the stored events.

13. References

1. Microchip Technology Inc., PIC16F87XA Data Sheet.
2. Maxim Integrated, DS1307 64 x 8 Serial Real-Time Clock Data Sheet.
3. Hitachi, HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver Data Sheet.
4. Microchip Technology Inc., MPLAB XC8 C Compiler User's Guide.
5. NXP Semiconductors, I2C-bus Specification and User Manual (UM10204).